

Handling Aperiodic Overloads — Buttazzo

Milestone 1: Technical Understanding and Research Framing

Rionela Kovaci

Hamm-Lippstadt University of Applied Sciences

Real-Time Systems Seminar

rionela.kovaci@stud.hshl.de

I. PROBLEM STATEMENT AND MOTIVATION

In real-time systems, tasks must complete their execution before a given deadline. Missing a deadline in a safety-critical system, such as a robot controller or an automotive braking system, is not just a performance issue but can cause serious or even catastrophic consequences [2].

The Earliest Deadline First (EDF) scheduling algorithm is widely used in dynamic real-time systems and has been proven to be optimal under normal load conditions [4]. EDF always assigns the processor to the task with the nearest deadline, preempting lower-priority tasks when necessary. As long as the total processor utilization does not exceed 100%, EDF guarantees that all tasks meet their deadlines.

However, EDF has a critical weakness in overload conditions. When too many tasks arrive at once and the total demanded CPU time exceeds what is available, EDF behaves in a completely unpredictable manner. Consider a simple example: three tasks τ_1 , τ_2 , τ_3 are active. If τ_3 experiences an execution overrun, meaning it takes longer than its worst-case estimate, its execution delays τ_2 , which in turn delays τ_1 . Under EDF, this cascading delay can propagate through the entire task set, causing all tasks to miss their deadlines even if only one task was responsible for the original overrun. This behavior is known as the *domino effect* [1]. In the worst case, the effective throughput of the system approaches zero.

Aperiodic tasks make this problem significantly harder. Unlike periodic tasks whose arrival times are known in advance and can be analyzed offline, aperiodic tasks arrive at irregular and unpredictable times. They can arrive in sudden bursts at any moment, creating unexpected overload conditions that the scheduler cannot anticipate. A scheduling algorithm designed only for normal load conditions is therefore not sufficient for systems that must also handle aperiodic task overloads in a safe and predictable way [1].

The core problem addressed in this work is therefore: how can a real-time system handle aperiodic task overloads gracefully, guaranteeing the execution of the most critical tasks while degrading performance in a controlled and predictable way, rather than collapsing entirely under load?

II. CORE TECHNICAL IDEA

Buttazzo and Stankovic propose the RED algorithm — Robust Earliest Deadline, as a robust extension of EDF that handles overload conditions without catastrophic failure [1].

The key insight is that instead of blindly accepting all tasks and risking a domino effect, the system should actively manage the load by classifying tasks by importance, detecting overload early, and selectively rejecting the least important tasks to protect the most critical ones.

RED combines three main mechanisms:

Task classification. Tasks are divided into two classes. **HARD** tasks are guaranteed to meet their deadlines only when the system is not overloaded. **CRITICAL** tasks are guaranteed to meet their deadlines even during overload conditions. This separation ensures that the most important tasks, for example safety-relevant control loops, always receive their required CPU time regardless of the current system load.

Rejection policy. When an overload is detected, RED removes the least important (lowest value) non-critical tasks from the ready queue to free up enough CPU time for the remaining tasks. The system does not simply reject the new arriving task; instead it searches for the combination of existing low-value tasks whose removal would resolve the overload with the least loss of value. Rejected tasks are not permanently discarded but placed in a dedicated Reject Queue for possible re-acceptance later.

Resource reclaiming. Whenever a task completes before its estimated worst-case execution time, the saved CPU time is immediately used to attempt re-accepting previously rejected tasks, ordered by their value. This makes the system adaptive: if the actual load turns out to be lower than the worst-case estimate, rejected tasks get a second chance. This mechanism significantly reduces unnecessary task losses.

The combination of these three mechanisms produces a system that degrades gracefully under overload, losing the least valuable work first, while keeping **CRITICAL** tasks always protected, even in extreme load situations [1].

III. RELEVANT FORMALISM, ALGORITHM, AND MODEL

A. Task Model

Each aperiodic task J_i is fully characterized by four parameters [1]:

- C_i — worst-case execution time (WCET)
- D_i — relative deadline
- m_i — deadline tolerance: the maximum time the task may finish after its deadline and still produce a valid result

- v_i — task value: the relative importance of the task compared to others in the set

The absolute deadline is $d_i = a_i + D_i$, where a_i is the arrival time. Tasks with maximum value $v_i = V_{max}$ are classified as CRITICAL. All other tasks are classified as HARD. The remaining execution time at the current moment is denoted c_i (dynamic computation time).

B. Schedulability Test

The schedulability of the task set is checked using the concept of *residual time* R_i , which represents how much slack remains between the estimated finishing time of task J_i and its deadline. Tasks are ordered by increasing deadline. The residual time is computed recursively as [1]:

$$R_1 = d_1 - t - c_1 \quad (1)$$

$$R_i = R_{i-1} + (d_i - d_{i-1}) - c_i \quad i > 1 \quad (2)$$

where t is the current time. A task J_i is guaranteed to complete within its deadline if and only if $R_i \geq 0$. The full task set is schedulable if and only if $R_i \geq 0$ for all J_i . This test runs in $O(n)$ time, making it efficient enough for online use at every task arrival.

Example. Consider two tasks at time $t = 0$: J_1 with $c_1 = 3$, $d_1 = 5$, and J_2 with $c_2 = 2$, $d_2 = 8$. Then: $R_1 = 5 - 0 - 3 = 2 \geq 0$ (guaranteed), $R_2 = 2 + (8 - 5) - 2 = 3 \geq 0$ (guaranteed). If a third task J_3 arrives with $c_3 = 4$, $d_3 = 6$, inserting it between J_1 and J_2 would give $R_3 = 2 + (6 - 5) - 4 = -1 < 0$, indicating an overload — J_3 cannot be accepted without rejection of another task.

C. Load and Overload Detection

The processor load in the interval $[t_a, d_i]$ is [1]:

$$\rho_i(t_a) = \frac{\sum_{d_k \leq d_i} c_k}{d_i - t_a} = 1 - \frac{R_i}{d_i - t_a} \quad (3)$$

The system is underloaded when $\rho_{max} \leq 1$ and overloaded when $\rho_{max} > 1$. The *Exceeding Time* of task J_i is defined as:

$$E_i = \max(0, -(R_i + m_i)) \quad (4)$$

This measures how much J_i will exceed its deadline even accounting for its tolerance m_i . The Maximum Exceeding Time $E_{max} = \max_i(E_i)$ gives the global severity of the overload.

D. RED Algorithm

When a new task J_a arrives, the RED guarantee algorithm proceeds as follows [1]:

- 1) Insert J_a into the task list ordered by deadline
- 2) Recompute residual times R_i for all tasks with deadline $\geq d_a$
- 3) Compute the Maximum Exceeding Time E_{max}
- 4) If $E_{max} = 0$: accept J_a (no overload)
- 5) If $E_{max} > 0$: apply rejection policy — find the set J^* of least-value non-critical tasks whose removal makes $E_{max} = 0$

- 6) If J^* is not empty: reject tasks in J^* , accept J_a
- 7) If J^* is empty: reject J_a (overload cannot be resolved)

Note that CRITICAL tasks already in the system can never be rejected, regardless of the severity of the overload.

IV. ASSUMPTIONS AND SCOPE LIMITATIONS

The RED algorithm operates under the following assumptions [1]:

- **Single processor only.** The algorithm is designed for uniprocessor systems. Extension to multiprocessor or multicore systems is not addressed and is mentioned only as future work.
- **Preemptive EDF scheduling.** The algorithm requires preemptive scheduling. Non-preemptive systems or fixed-priority schedulers such as Rate Monotonic are not supported.
- **Arrival times unknown.** The arrival time of each aperiodic task is not known in advance. The guarantee test must therefore be performed online at each arrival.
- **WCET known at arrival.** The worst-case execution time C_i must be known at the moment the task arrives. In practice this is often obtained by static analysis of the task code, but may not always be available or accurate.
- **Independent tasks.** No shared resources or precedence constraints between tasks are considered. The authors acknowledge this as a limitation and mention resource sharing as a direction for future work.
- **Aperiodic tasks only.** The model handles only aperiodic tasks. The interaction with a set of periodic tasks running concurrently, as is typical in embedded systems using AUTOSAR or FreeRTOS, is not analyzed.

These assumptions are restrictive for real embedded systems. In practice, most embedded platforms such as automotive ECUs run a mix of periodic and aperiodic tasks, often on resource-constrained hardware where maintaining four runtime queues may introduce non-trivial overhead.

V. RUNTIME, MEMORY, AND RESOURCE ASPECTS

Computational complexity. The guarantee test has $O(n)$ complexity, where n is the number of currently active tasks. This is efficient and suitable for online execution at each task arrival [1].

Queue management. The RED framework maintains four queues at runtime [1]:

- **Ready Queue** — tasks accepted and waiting for CPU time, ordered by deadline (EDF order)
- **Reject Queue** — tasks rejected due to overload, ordered by decreasing value, kept for possible re-acceptance
- **Miss Queue** — tasks whose laxity has become negative and can no longer be re-accepted; records late tasks
- **Term Queue** — tasks that completed successfully within their timing constraints

The Miss and Term Queues serve as a history log for debugging and system monitoring. The Reject Queue is the most operationally critical: it must be scanned at every task completion to check for re-acceptance opportunities.

Memory considerations. The paper does not analyze memory consumption in detail. For systems with very limited RAM, such as small microcontrollers with tens of kilobytes, maintaining these four queues and the full task parameter set (C_i, D_i, m_i, v_i) for every rejected task may not always be feasible without careful memory management.

Overhead of resource reclaiming. The reclaiming mechanism requires re-scanning the Reject Queue at every task completion, which adds overhead proportional to the number of rejected tasks. In heavily overloaded systems this overhead could become significant.

VI. UNCLEAR POINTS AND OPEN QUESTIONS

- How does RED interact with a concurrent set of periodic tasks? In most real embedded systems both periodic and aperiodic tasks run simultaneously, but this combination is not analyzed in the paper.
- How should the task value v_i be assigned in practice? The paper assumes values are given as input but provides no guidance on how to define them for a real application.
- How should the deadline tolerance m_i be chosen? A tolerance that is too small causes unnecessary rejections; one that is too large may allow tasks to finish too late to be useful.
- Can the algorithm be extended to multiprocessor systems? This is mentioned as future work but not addressed.
- What happens when the Reject Queue itself becomes full? No overflow policy for the Reject Queue is discussed.

REFERENCES

- [1] G. C. Buttazzo and J. A. Stankovic, "Adding Robustness in Dynamic Preemptive Scheduling," in *Responsive Computer Systems: Steps Toward Fault-Tolerant Real-Time Systems*, Kluwer Academic Publishers, 1995.
- [2] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3rd ed. Springer, 2011.
- [3] M. Spuri and G. C. Buttazzo, "Scheduling Aperiodic Tasks in Dynamic Priority Systems," *Real-Time Systems*, vol. 10, pp. 179–210, 1996.
- [4] C. L. Liu and J. W. Layland, "Scheduling Algorithms for Multiprogramming in a Hard Real-Time Environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.